

Week 1 - Day 1a Supplementary: Operations and Data Types

Exercise 1: More Order of Operations []

Compute the following expressions by hand. Feel free to use Python as a calculator to check your answers.

NOTE: *You can use Use this cell if you would like to use Python as a calculator for the following questions.*

1.1 `10 * 23 / 5`

1.2 `(1 + 2) * 3`

1.3 `1 + (2 * 3.0)`

1.4 `12 * 5 / 4`

1.5 `12 * 5.0 // 7`

Exercise 2: Invalid Operations []

2.1 Fix the following expression. The expected result is `340.0`

```
3 * * 2 / / 2 + 32/2 x 21
```

TODO: *Your code here*

2.2 Fix the following expression. The expected result is `16.0`

```
8.0 * * 2 // 2 ** (2 - 3) / 8
```

TODO: *Your code here*

Exercise 3: Use Python As a Calculator []

3.1 Compute the following expression in Python by hand, which is the result?

```
(3 // 2 ** 2 + 3) / 5
```

1. `0.75`
2. `0.8`
3. `0.6`

4. 0

3.2 Which of the following Python expressions is equivalent to the following expression?

```
342 // 12 + 23 ** 1 + 23 - 762
```

1. `342 // 12 + 23 ** (1 + 23 - 762)`
2. `342 // (12 + 23) ** 1 + 23 - 762`
3. `342 // 12 + (23 ** 1) + 23 - 762`
4. `342 // 12 + 23 ** (1 + 23) - 762`

Exercise 4: Modulus [] [] [] []

In 1a, we explored how the modulo operator works when both arguments are positive. The modulo operator can also be extended to negative numbers. From the official Python spec on the [modulo operator](#):

The % (modulo) operator yields the remainder from the division of the first argument by the second. The numeric arguments are first converted to a common type. A zero right argument raises the ZeroDivisionError exception. The arguments may be floating point numbers, e.g., `3.14%0.7` equals `0.34` (since `3.14` equals `4*0.7 + 0.34`.) **The modulo operator always yields a result with the same sign as its second operand (or zero); the absolute value of the result is *strictly smaller* than the absolute value of the second operand.**

4.1 What is `-1 % 3`

4.2 What is `4 % -3`

4.3 What is `2.1 % 3.1`?

4.4 Which of the following options have the same values as the following one?

```
4233 % 372 // 31 % 31 ** 3
```

1. `4233 % 372 // 31 % (31 ** 3)`
2. `(4233 % 372 // 31 % 31) ** 3`
3. `4233 % (372 // 31) % 31 ** 3`
4. `4233 % 372 // (31 % 31) ** 3`

4.5 Suppose that you have three functions f , g and k , defined in Python as:

```
f = lambda x, y: x % y
g = lambda x, y: x - x // y * y
k = lambda x, y: x - y * x / y
```

Important Note: The Python syntax `f = lambda x: 1` is equivalent to $f(x)=1$. For example, the function `k = lambda x, y: x - y * x / y` above is equivalent to

$$k(x, y) = x - (y * x) / y.$$

Assume that both $x, y > 0$, which two are equivalent in Python syntax?

4.6 Extract the middle three digits from the following number `451987382`

Hint: Use the modulus operator `%` and integer division `//`

```
# TODO: Your code here
```

4.7 Update the middle three digits from the following number `451,987,382` to `123`. Use modulus operation.

Hint: Use the modulus operator `%` and integer division `//`

```
# TODO: Your code here
```

Exercise 5: Taylor Series [] [] [] []

In mathematics, the Taylor series or Taylor expansion of a function is an infinite sum of terms that are expressed in terms of the function's derivatives at a single point. The n -th order Taylor expansion of f at point a is given by the formula:

$$f(x) = f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots$$

where $f'(x) = \frac{\partial f(x)}{\partial x}$.

For example, the 3rd-order approximation of e^x at point $a=0$ is

$$e^x = e^0 + \frac{e^0}{1!}(x) + \frac{e^0}{2!}(x)^2 + \frac{e^0}{3!}(x)^3$$

since $\frac{\partial e^x}{\partial x} = e^x$.

Euler's Number: Euler's number e is a mathematical constant that is the base of the natural logarithm: the unique number whose natural logarithm is equal to one. It is approximately equal to 2.718. Use this value for your computations.

Factorials: For any positive integer n , $n!$ is calculated as $n * (n - 1) * (n - 2) * \dots * 2 * 1$. For example, $4! = 4 * 3 * 2 * 1 = 24$

5.1 Compute the $e^{0.3}$.

```
# TODO: Your code here
```

5.2 Now, compute the 3rd-order approximation of $e^{0.3}$.

```
# TODO: Your code here
```

5.3 Given the 3rd-order expansion of e^x at point 0, calculate the accuracy percentage of $e^{0.3}$.

```
# TODO: Your code here
```